

TRATADO CANÔNICO DO ECOSSISTEMA AIDD

Engenharia Agêntica Autônoma, A Tríade Canônica de Criação e os 24 Quality
Gates Determinísticos

Versão: 3.0 Canônica **Data:** 18 de Setembro de **Status:** 100% Homologado
2026

Sumário Executivo

1. Preâmbulo e Princípios Fundamentais	2
As Leis Invioláveis da Governança AIDD	2
2. As 8 Ferramentas Homologadas (tools/)	3
3. A Tríade Canônica de Criação	3
Fluxo 01 — aidd-pure (Do Zero Puro)	3
Fluxo 02 — aidd-open (Motores Open-Source)	3
Fluxo 03 — aidd-bridge (Low-Code Desatado)	4
4. O Funil Universal de Convergência	4
5. Contratos Formais de Handoff e Validação JSON Schema	4
6. Matriz dos 24 Quality Gates Determinísticos	5
7. Guia Operacional: Dev Sênior vs. Extremo Leigo	6
Para o Desenvolvedor Sênior (Terminal & CLI)	6
Para o Usuário Final ou Foco em Negócio (Slash Commands & Chat)	6
8. Conclusão e Homologação	7

1. Preâmbulo e Princípios Fundamentais

O **Ecossistema AIDD (Artificial Intelligence Driven Development)** é uma plataforma unificada de engenharia agêntica de software projetada para erradicar definitivamente o paradigma amador do “*Vibe Coding*” — a geração cega de código sem testes, acoplada a dependências proprietárias, com alucinações de modelos de linguagem e falsas alegações de conformidade.

O ecossistema estabelece uma barreira de engenharia mecânica onde a IA generativa é estritamente contida por **Quality Gates binários determinísticos**, **Fatias Verticais isoladas (Vertical Slice Architecture)** e **contratos formais de handoff baseados em JSON Schema**.

As Leis Invioláveis da Governança AIDD

Lei #1: Determinismo Primeiro (Zero Token Fallacy)

Toda validação, checagem estrutural, linting, análise sintática (AST) e auditoria de segurança é executada por compiladores e scripts mecânicos determinísticos (Python puro, Hadolint, Checkov, Pytest). A inteligência artificial nunca audita a si mesma com texto livre.

Lei #2: Qualidade Binária

Qualquer verificação gera apenas dois estados: `exit 0` (aprovado e promovido) ou `exit 1` (bloqueado sumariamente com fail-fast). Não existem avisos toleráveis em produção.

Lei #5: Zero Stubs / Zero Mocks em Produção

Todo software produzido nasce com 100% de contratos tipados, banco de dados real em execução concorrente e suíte automatizada de testes reais. Proibido o uso de `TOD0`, `pass`, mocks estáticos ou retornos fictícios em código de produção.

Lei #11: Padrão-Ouro de Stack Tecnológica

Salvo especificação contrária expressa no plano de intake:

- **Frontend:** Next.js 14 (App Router) + TypeScript + Tailwind CSS + Lucide Icons + Zod.
- **Backend:** Python puro estruturado em Clean Architecture + SQLite em modo WAL concorrente (ou PostgreSQL nativo) + FastAPI com OpenAPI 3.1 nativo.

Lei #10: Quarteto Sine Qua Non Dinâmico

Todo sistema gerado nasce nativamente com quatro pilares autônomos que cobrem 100% dos módulos da aplicação:

1. **Swagger Studio** (`/swagger` ou `/docs`): Especificação interativa OpenAPI 3.1.
2. **Webhook Studio** (`/webhooks`): Gestão, teste e disparo de webhooks assíncronos.
3. **MCP Studio** (`/mcp`): Exposição padronizada via Model Context Protocol.
4. **Guia do Utilizador** (`/guia` ou `/docs/guia`): Manual vivo da aplicação para utilizadores finais.

2. As 8 Ferramentas Homologadas (tools/)

O ecossistema é modularizado em 8 ferramentas atômicas com fronteiras de responsabilidade matematicamente delimitadas:

Ferramenta	Diretório	Entrada Principal	Entrega Primária
AIDD Forge	tools/aidd-forge	Diretório alvo / CLI	Governança, regras e hooks pre-commit.
AIDD Planner	tools/aidd-planner	Linguagem natural / Requisitos	Planta baixa BDD/SDD e PLANNER.json.
AIDD Generator	tools/aidd-generator	PLANNER.json estruturado	Código do zero puro com TDD Red-Green.
AIDD Factory	tools/aidd-factory	Motores OSS + Plano	Gateway BFF FastAPI e Compose unificado.
AIDD Bridge	tools/aidd-bridge	Export Lovable/v0/Bolt	Desacoplamento de BaaS e Postgres nativo.
AIDD Master	tools/aidd-master	Código bruto das Engines	Monólito VSA, Next.js 14 e OpenAPI.
AIDD Enterprise	tools/aidd-enterprise	Código do Master	Blindagem criptográfica SHA-256 e RBAC.
AIDD Ops	tools/aidd-ops	Código Enterprise	Provisionamento VPS, Docker Swarm e SSL.

3. A Tríade Canônica de Criação

O ecossistema resolveu definitivamente as sobreposições de ferramentas estabelecendo que qualquer software origina de uma fundação de governança (**Forge**) e um intake estruturado (**Planner**), derivando para um dos três motores especializados da Tríade:

Fluxo 01 — aidd-pure (Do Zero Puro)

- **Pipeline:** [FORGE -> PLANNER] -> GENERATOR -> [MASTER -> ENTERPRISE -> OPS]
- **Comando Slash:** /pure <ideia>
- **Comando CLI:** python ecossistema.py run-fluxo --fluxo pure --nome "<nome>"
- **Aplicação:** Software exclusivo, regras de negócio inovadoras ou soluções de alta densidade algorítmica.
- **Rigor:** Aplica TDD Red-Green estrito ao longo de um pipeline de 8 fases com auto-crítica e métricas de qualidade.

Fluxo 02 — aidd-open (Motores Open-Source)

- **Pipeline:** [FORGE -> PLANNER] -> FACTORY -> [MASTER -> ENTERPRISE -> OPS]
- **Comando Slash:** /open <ideia>
- **Comando CLI:** python ecossistema.py run-fluxo --fluxo open --nome "<nome>"
- **Aplicação:** Sistemas empresariais que aproveitam componentes maduros (ERPs, CRMs, automação de fluxos, bots).

- **Rigor:** Curadoria de engines consagradas, isolamento em contêineres e geração automática de Gateway BFF FastAPI.

Fluxo 03 — aidd-bridge (Low-Code Desatado)

- **Pipeline:** [FORGE -> PLANNER] -> BRIDGE -> [MASTER -> ENTERPRISE -> OPS]
- **Comando Slash:** /bridge <pasta> <nome>
- **Comando CLI:** python ecossistema.py run-fluxo --fluxo bridge --origem <pasta>
- **Aplicação:** Resgate e autonomia de protótipos criados em Lovable, Bolt ou v0.
- **Rigor:** Scanner estático de código, conversão determinística de SQL para Postgres nativo e preservação total da interface React concebida.

4. O Funil Universal de Convergência

Independente de qual motor construiu a base do sistema, todos os fluxos convergem mandatoriamente para a mesma esteira de homologação e produção:

1. **AIDD Master:** Converte e organiza a solução em Fatias Verticais desacopladas (**Vertical Slice Architecture**), gera o Frontend Next.js 14 App Router nativo e injeta o Quarteto Sine Qua Non.
2. **AIDD Enterprise:** Aplica a auditoria criptográfica de componentes críticos (RBAC com Argon2id, Transactional Outbox, Rate Limiting distribuído, sanitização OWASP) e sela os hashes SHA-256 no manifesto.
3. **AIDD Ops:** Calcula o sizing de hardware, conecta à VPS remota via SSH criptografado, provisiona os contêineres Docker sob Traefik com SSL automático Let's Encrypt e configura monitoramento contínuo via Uptime Kuma.

5. Contratos Formais de Handoff e Validação JSON Schema

A integridade do ecossistema é mantida através de schemas JSON estritos localizados em componentes/compartilhado/specs/:

Transição	Arquivo Schema	Campos e Invariantes Auditados
Planner → Engine	handoff-planner-to-engine.schema.json	Metadados do projeto, cenários BDD obrigatórios, dicionário formal de entidades e declaração do Quarteto.
Engine → Master	handoff-engine-to-master.schema.json	Rotas REST, schemas Pydantic, DDL SQL de banco e relatório de testes reais verdes da engine.
Master → Enterprise	handoff-master-to-enterprise.schema.json	Estrutura VSA limpa, interfaces Next.js geradas e rotas operacionais ativas do Quarteto.
Enterprise → Ops	handoff-enterprise-to-ops.schema.json	Manifesto com hashes SHA-256, lockfiles auditados, credenciais blindadas e parâmetros de sizing VPS.

6. Matriz dos 24 Quality Gates Determinísticos

O ecossistema é protegido por 24 portões de qualidade mecânicos executados automaticamente via pre-commit ou agregados via `python ecossistema.py audit`:

Gate	Script Verificador	Alvo de Auditoria Mecânica
G1	G_ORQUESTADOR_SINCRONO.py	Audita a CLI unificada e conformidade estrita com os schemas formais de handoff.
G2	G_TESTES_REAIS.py	Executa pytest real em todas as 7 ferramentas (2.304 testes monitorados). Reprova com 1 falha.
G3	G_HARNESS_COMPAT.py	Verifica sincronismo bidirecional de skills e comandos em todos os 6 harnesses de IA.
G4	G_ECOSSISTEMA_INTEGRIDADE.py	Varredura de sintaxe AST Python e integridade de todos os subprojetos.
G5	G_DRIFT_NUCLEO_COMPARTILHADO.py	Impede deriva de código entre bibliotecas compartilhadas do Master e Enterprise.
G6	G_DEPENDENCIAS_PIN_HASH.py	Bloqueia dependências sem pinagem estrita (==) ou lockfiles sem hash SHA-256.
G7	G_FRONTEND_LAYERS.py	Audita frontend Next.js e proíbe chamadas de rede direta em componentes de UI pura.
G8	G_ISOLATION_AUDIT.py	Audita isolamento VSA via AST e proíbe imports cruzados entre fatias verticais.
G9	G_HADOLINT.py	Audita conformidade OCI e melhores práticas em todos os 17 Dockerfiles do projeto.
G10	G_INFRA_COMPOSE.py	Audita segurança de portas, redes e volumes em arquivos docker-compose (Checkov).
G11	G_PROTOCOL_FALLBACK.py	Garante paridade total: nenhuma rota existe no MCP sem contrapartida OpenAPI/REST.
G12	G_LLM_PROMPT_SHIELD.py	Audita sanitização obrigatória de prompts contra Prompt Injection em clientes LLM.
G13	G_DRIFT_ANALYZER.py	Detecta redundância estrutural entre fatias VSA orientando extração canônica.
G14	G_PROTOTYPE_REWRITE.py	Garante que código de sandbox/ só seja promovido para produção com testes TDD.
G15	G_ZERO_HEADLESS.py	Bloqueia execução desassistida de subagentes ou subprocessos invisíveis paralelos.
G16	G_HONESTIDADE_ROTULO.py	Bloqueia saídas com termos de marketing que excedam o comprovado em testes reais.
G17	G_CLI_HELP_CONSISTENCIA.py	Compara flags citadas em mensagens contra parâmetros reais das CLIs argparse/click.
G18	G_COMPONENTE_AGNOSTICO.py	Audita integridade e paridade multi-harness de novos componentes agnósticos.

Gate	Script Verificador	Alvo de Auditoria Mecânica
G19	G_SEGREDOS.py	Escaneia repositório contra vazamento de credenciais e chaves via detect-secrets.
G20	G_ARQUITETURA_DELIVERABLE.py	Valida conformidade com Clean Architecture e DDD em entregáveis estruturados.
G21	G_ESCRITOR_ATOMICO.py	Audita o uso de gravação atômica para impedir corrupção em arquivos de estado.
G22	G_TRANSACTION_LOG_LRU.py	Audita deterministicamente o transaction log com cache LRU e testes espelhados.
G23	G_UNIVERSAL_HARNESS.py	Garante paridade e wiring universal de componentes em todos os ambientes.
G24	G_SUPPLY_CHAIN.py	Audita conformidade e integridade criptográfica da cadeia de dependências.

7. Guia Operacional: Dev Sênior vs. Extremo Leigo

O ecossistema foi concebido com uma interface dual perfeita:

Para o Desenvolvedor Sênior (Terminal & CLI)

Executar a Tríade Canônica de Criação:

```
python ecossistema.py run-fluxo --fluxo pure --nome "Tarefas SaaS" --slug tarefas --dominio prod
python ecossistema.py run-fluxo --fluxo open --nome "Clínica ERP" --slug clinica --dominio saude
python ecossistema.py run-fluxo --fluxo bridge --nome "Hub Delivery" --origem ./exports/lovable
```

Manutenção e Auditoria Global:

```
python ecossistema.py audit # Roda os 24 Quality Gates
python ecossistema.py status # Exibe saúde do ecossistema
python ecossistema.py components sync --tipo todos # Sincroniza os 6 harnesses
```

Para o Usuário Final ou Foco em Negócio (Slash Commands & Chat)

Criar aplicações completas sem tocar no terminal:

```
/pure "Construir um sistema de gestão escolar com controle de presenças e boletim"
/open "Criar uma plataforma de atendimento ao cliente multicanal com WhatsApp e N8N"
/bridge ./meu-projeto-lovable meu-sistema-saas
```

Comandos Rápidos e Refinamentos:

```
/master financeiro # Adiciona nova fatia vertical com tela e banco
/enterprise rbac autenticacao # Injeta blindagem empresarial SHA-256
/ops plan "Clínica médica" # Dimensiona infraestrutura VPS e prepara deploy
/aidd-orchestrator # Inicia o assistente interativo da Tríade Canônica
```

8. Conclusão e Homologação

O Ecosistema AIDD consolida o mais alto patamar de engenharia de software agêntica da atualidade. A fusão do planejamento BDD/SDD com a flexibilidade da Tríade Canônica, a convergência para o monólito modular Next.js e a proteção inegociável de 24 Quality Gates com mais de 2.300 testes reais garante que toda aplicação gerada seja **autônoma, determinística, soberana e pronta para produção desde o primeiro commit**.